

Beziehungs-Erhebung — von der Umfrage zur geltenden Matrix

Dieses Dokument beschreibt **haarklein**, wie die Beziehungen zwischen den Gruppierungen erhoben, abgeglichen und in die geltende Matrix übernommen werden — inklusive der Begründung für jede Design-Entscheidung. Wer verstehen will, **warum** etwas so gebaut ist und nicht anders, ist hier richtig.

Für die **statische Ausgangs-Matrix** (die von Hand gepflegten 34 Beziehungen) siehe [CREW_RELATIONS.md](#). Dieses Dokument beschreibt den **dynamischen Prozess**, mit dem die Spieler-Gruppierungen ihre Beziehungen selbst melden und daraus ein geltender Stand wird.

- **Seite:** <https://crime.bots.sektorrp.eu/beziehungen>
 - **Backend:** [backend/routes_relations_survey.py](#)
 - **Frontend:** [frontend/rerelations.html](#) + `relationsSurvey()` in [frontend/app.js](#)
 - **Bot:** [backend/bot.py](#) (Dropdown-Menüs + Auswertung)
 - **Prompt:** [backend/prompts.py](#) (`RELATION_ARBITRATION_*`)
-

Inhalt

1. [Das Grundproblem](#)
 2. [Die zwei Tabellen — und warum es zwei sind](#)
 3. [Der Gesamttablauf in fünf Schritten](#)
 4. [Schritt 1 — Erheben \(Discord-Dropdowns\)](#)
 5. [Schritt 2 — Korrigieren & Nachtragen](#)
 6. [Schritt 3 — Der Befund \(einig/abweichend/Widerspruch\)](#)
 7. [Schritt 4 — KI-Schiedsspruch](#)
 8. [Schritt 5 — Übernehmen in die geltende Matrix](#)
 9. [Wie die Begründung in die Aufträge fließt](#)
 10. [Der Nachtrags-Modus \(Neuzugänge & Erinnerungen\)](#)
 11. [Aufräumen & Löschen](#)
 12. [API-Referenz](#)
-

1. Das Grundproblem

Jede Gruppierung soll melden, wie sie zu den anderen steht. Aber:

- **Die Spieler kennen sich nicht alle** und können sich nicht absprechen. Die Yardis halten die Yakuza für verfeindet, die Yakuza die Yardis für Geschäftspartner — beides gleichzeitig ist im Spiel unmöglich.
- **Es braucht am Ende genau eine geltende Beziehung pro Paar**, sonst kommt es zu „komischen Momenten“: Man beleidigt jemanden, der einen für befreundet hält.
- **Der finale Stand muss vor dem Start feststehen** und jeder muss ihn kennen —

aber niemand darf erfahren, was die Gegenseite **gewünscht** hatte (das wäre Spieler-Wissen, das die Figur nicht hat).

Die Lösung trennt sauber zwischen **Wunsch** (was jede Seite meldet) und **Entscheidung** (was gilt). Genau das spiegelt sich im Datenmodell.

2. Die zwei Tabellen — und warum es zwei sind

relation_proposals — **die Wünsche (gerichtet)**

Jede Zeile ist **eine gerichtete Einschätzung**: „Gruppierung A sieht Gruppierung B als X“. $A \rightarrow B$ und $B \rightarrow A$ sind zwei getrennte Zeilen.

Spalte	Bedeutung
<code>from_crew_id, to_crew_id</code>	Richtung der Einschätzung
<code>relation_type</code>	ALLIED / BUSINESS / NEUTRAL / RIVAL / HOSTILE
<code>discord_user_id, discord_user_name</code>	wer geklickt hat (oder „Hand-Korrektur“)
<code>updated_at</code>	letzte Änderung

`UniqueConstraint(from_crew_id, to_crew_id)` → pro Richtung genau eine Zeile, eine neue Auswahl überschreibt die alte.

Warum gerichtet? Genau die Schieflage — A hält B für Partner, B sieht das anders — ist das wertvolle Material. Würde man nur einen Wert pro Paar speichern, ginge diese Spannung verloren, bevor man sie überhaupt gesehen hat. Die Gegenüberstellung lebt davon, beide Richtungen nebeneinander zu zeigen.

crew_relations — **der geltende Stand (symmetrisch)**

Die **eine** Beziehung, die zwischen zwei Gruppierungen tatsächlich gilt. Kanonisch mit `crew_a_id < crew_b_id`, damit jedes Paar nur einmal existiert. Diese Tabelle — und **nur diese** — wird von der Auftragsgenerierung und der Story gelesen (`prompts.py`, `build_user_prompt()` u.a.).

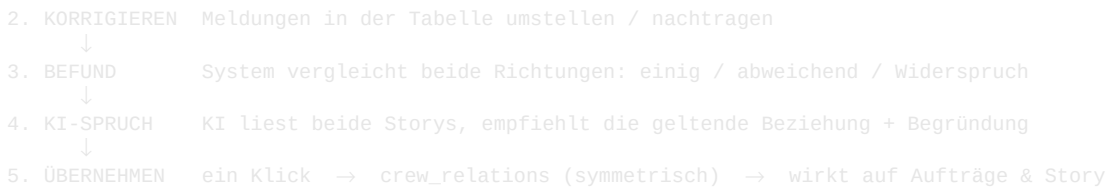
Spalte	Bedeutung
<code>crew_a_id, crew_b_id</code>	Paar, immer $a < b$
<code>relation_type</code>	die geltende Beziehung
<code>notes</code>	das WARUM — fließt in die Aufträge ein (siehe §9)

Warum symmetrisch und nicht auch gerichtet? Das gesamte bestehende System (Aufträge, Story, KI-Enrich, Gang-Detailseite) rechnet mit einem Wert pro Paar. Ein Wechsel auf gerichtete geltende Beziehungen wäre ein Eingriff in all diese Konsumenten — für wenig Gewinn, da im Spiel ohnehin **eine** Wahrheit pro Paar gilt. Die Asymmetrie lebt in *relation_proposals* weiter, wo sie hingehört.

Merksatz: *relation_proposals* = Wünsche (gerichtet, dürfen sich widersprechen). *crew_relations* = Entscheidung (symmetrisch, gilt fürs Spiel).
Der Übergang zwischen beiden ist **immer ein bewusster Klick**, nie automatisch.

3. Der Gesamtablauf in fünf Schritten

1. ERHEBEN Dropdown-Umfrage im Discord → *relation_proposals* (gerichtet)
↓



Jeder Schritt schreibt nur in `relation_proposals` — **außer Schritt 5**, der als einziger `crew_relations` anfasst. Das ist die bewusste Grenze zwischen „gesammelt“ und „gilt“.

4. Schritt 1 — Erheben (Discord-Dropdowns)

Über **Beziehungen** → **Umfrage versenden** postet der Bot in den Auftrags-Channel jeder aktiven Gruppierung eine Nachricht mit **Auswahlmenüs** — eines pro andere Gruppierung. Bei 14 aktiven Gruppierungen bewertet jede also 13 andere.

Warum Dropdowns und nicht Freitext?

Die entscheidende Ersparnis: **Die Antwort kommt strukturiert an** — als `crew_id` + `relation_type`, nicht als Freitext, den erst jemand parsen müsste. „Geschäft“, „business“, „geschäftl.“ und „Russische Mafia“ vs. „Russian Mafia“ — jede dieser Ungenauigkeiten wäre eine mögliche Fehlzusammenordnung. Beim Dropdown gibt es sie nicht.

Technische Details

- **Max. 5 Menüs pro Nachricht** (Discord-Limit für Bedienzeilen). Bei mehr wird auf mehrere Nachrichten aufgeteilt (`SELECTS_PER_MESSAGE = 5` in `bot.py`).

- **Der Gruppenname steht in JEDER Option**, nicht nur im Platzhalter. Grund:

Discord ersetzt den Platzhalter, sobald etwas gewählt ist. Stünde der Name nur dort, zeigten alle Menüs danach bloß noch „geschäftlich“ — welche Gruppierung gemeint ist, wäre nicht mehr erkennbar, und ein Ändern würde zum Raten.

- **Die eigene Gruppierung taucht nie auf** — die Zielliste wird pro Empfänger gebaut und die eigene ID herausgefiltert.

- **Auswertung über das rohe `on_interaction`-Event**, nicht über View-Callbacks.

Grund: Persistente Views müssten nach jedem Bot-Neustart neu registriert werden; bei einem vergessenen Reregister wären die Menüs nach einem Deploy tot. Das rohe Event kommt immer an, überlebt also Neustarts.

- **Stille Quittung** (`interaction.response.defer()`): Discord verlangt binnen

3 Sekunden eine Antwort, sonst „Interaktion fehlgeschlagen“. Der Bot bestätigt unsichtbar — die Rückmeldung übernimmt das Menü selbst, in dem die getroffene Wahl stehen bleibt. Kein „Notiert“-Spam im Channel.

Frist / Countdown

Optional lässt sich eine Frist anhängen — als **Discord-Zeitstempel** (`<t:UNIX:F>` für Datum, `<t:UNIX:R>` für den mitlaufenden Countdown). Vorteil: Jeder Leser sieht die Zeit in **seiner** Zeitzone, der Countdown läuft live. Wählbar als fester Zeitpunkt oder als Dauer ab Versand — die Dauer wird **serverseitig** in einen Zeitstempel umgerechnet, damit eine falsch gestellte Browser-Uhr die Frist nicht verschiebt.

5. Schritt 2 — Korrigieren & Nachtragen

In der **Gegenüberstellung** ist jede der beiden linken Bewertungen ein **Dropdown**. Umstellen ändert die Meldung sofort (**PUT** `/proposal`, Upsert):

- **Korrigieren:** Hat jemand falsch geklickt, stellst du den Wert um.
- **Nachtragen:** Steht dort „— offen —“ (die Gruppierung hat nicht geantwortet),

setzt du selbst einen Wert. So füllst du offene Richtungen von Hand, ohne auf eine Discord-Antwort zu warten.

Hand-Korrekturen werden intern als `discord_user_name = "Hand-Korrektur"` markiert — damit später nachvollziehbar bleibt, welche Werte von der Spielleitung kamen und welche aus dem Discord.

Das bleibt alles **gerichtet** und betrifft nur `relation_proposals`. Die geltende Matrix ist noch unberührt.

6. Schritt 3 — Der Befund (einig/abweichend/Widerspruch)

Sobald **beide** Richtungen eines Paares vorliegen, vergleicht das System sie auf einer Skala:

`verbündet(0) → geschäftlich(1) → neutral(2) → rivalisierend(3) → feindlich(4)`

Der **Abstand** der beiden Werte auf dieser Skala bestimmt den Befund:

Abstand	Befund	Bedeutung
—	offen	mindestens eine Seite hat noch nicht geantwortet
0	einig	beide sagen dasselbe
1	abweichend	benachbart, unkritisch (z.B. verbündet/geschäftlich)
≥ 2	Widerspruch	echter Konflikt, der eine Entscheidung braucht

Warum diese Skala? Sie ordnet die fünf Typen von „engste Bindung“ bis „offene Feindschaft“. Dadurch ist „benachbart“ (verbündet vs. geschäftlich) messbar harmloser als „weit auseinander“ (verbündet vs. feindlich). Die Sortierung stellt die Widersprüche nach oben — das ist die Arbeitsliste.

7. Schritt 4 — KI-Schiedsspruch

Der **KI-Knopf** pro Paar liest die **Hintergrund-Story** und das **kriminelle Geschäft beider Gruppierungen** sowie die abgegebenen Sichten und empfiehlt die eine geltende Beziehung mit kurzer Begründung (**POST** `/ai-suggest`).

Wie die KI entscheidet — der Leitgedanke

Der System-Prompt (`RELATION_ARBITRATION_SYSTEM_PROMPT`) gibt vor: **nach Dramaturgie entscheiden, nicht per Kompromiss**. Konkret:

- **Konkurrierende Geschäfte** im selben Feld → eher Rivalität oder Feindschaft.
- **Sich ergänzende Geschäfte** → geschäftliche Nähe.
- **Einseitig behauptete Freundschaft**, die die Gegenseite nicht teilt, ist meist keine — sie kippt eher in Rivalität.

- **Neutral** nur, wenn die beiden wirklich nichts verbindet oder trennt.

Beispiel aus dem echten Test: Yakuza sieht Los Aztecas als **rivalisierend**, die Aztecas sehen die Yakuza als **verbündet** (Abstand 3 → Widerspruch). Die KI wählte **rivalisierend**, mit der Begründung, dass die einseitige Bündnis-Sicht der Aztecas von der Yakuza nicht geteilt wird und die unterschiedlichen Geschäftsmodelle für Spannung sprechen.

Wichtig: Die KI schreibt nichts

`POST /ai-suggest` ist ein **reiner Vorschlag**. Er füllt das Dropdown „Finaler Stand“ und zeigt die Begründung als aufklappbare Zeile — geltend wird er erst durch **Übernehmen**. Die „vorher sehen, dann freigeben“-Logik bleibt gewahrt.

8. Schritt 5 — Übernehmen in die geltende Matrix

Der **Übernehmen**-Knopf schreibt die eine geltende Beziehung des Paares nach `crew_relations` (`POST /finalize`).

Das Dropdown „Finaler Stand“

Vorbelegt mit einem **Vorschlag**, in dieser Reihenfolge:

1. Wurde ein KI-Vorschlag geholt → dessen Wert.
2. Sonst der bereits gepflegte Wert (`current`), falls vorhanden.
3. Sonst der **Regel-Vorschlag**: bei Einigkeit der gemeinsame Wert, bei

Uneinigkeit der **härtere** (höhere Skalenstufe).

Warum bei Uneinigkeit der härtere Wert? Zwei Gründe: Im RP eskaliert ein Konflikt eher, als dass er sich in Wohlwollen auflöst — und der Vorschlag soll **auffallen**, nicht glätten. Er ist ein Startpunkt, keine Entscheidung; du stellst ihn jederzeit um.

Sicherheitsdetails

- **IDs werden sortiert** (`crew_a_id < crew_b_id`) — egal, in welcher Reihenfolge das Paar übergeben wird, es trifft immer dieselbe Zeile.
 - **Leerer Typ löscht** die Beziehung (zurück auf implizit neutral).
 - „**war: ...**“ erscheint klein neben dem Dropdown, wenn der gepflegte Wert vom Ziel abweicht — du siehst, was du überschreibst.
 - **Übernehmen ist deaktiviert**, solange nichts zu ändern ist.
 - Der Klick **fragt vorher** und weist darauf hin, dass es sofort auf Aufträge und Story wirkt.
-

9. Wie die Begründung in die Aufträge fließt

Das ist der Punkt, der den ganzen Aufwand auszahlt.

`crew_relations.notes` wird von der Auftragsgenerierung **bereits gelesen**. In `prompts.py` steht pro verbundener Gruppierung im Prompt:

- **Los Aztecas** (rival): <die Notiz>

Beim Übernehmen wird die **KI-Begründung automatisch als Notiz mitgespeichert** — aber nur, wenn der übernommene Wert auch dem KI-Wert entspricht (sonst passt die Begründung nicht zur gesetzten Beziehung). Dadurch:

- Die Auftrags-KI erfährt beim nächsten Auftrag nicht nur **dass** zwei Gruppen

rivalisieren, sondern **warum** — „alte Blutfehde ums Hafenrevier“ — und kann den Auftrag genau daran ausrichten.

- Änderst du später nur den Typ (ohne KI), bleibt die vorhandene Notiz erhalten

(**notes** = **None** im Request lässt sie unangetastet; **""** leert sie gezielt).

In der Tabelle zeigt ein ■ neben einem Paar an, dass eine Notiz hinterlegt ist — Maus drüber zeigt den Text.

Das ist die eigentliche Pointe: Die Beziehung ist nicht nur ein Etikett, sondern trägt ihren Grund mit sich. Der Grund steuert ab der Übernahme mit, welche Aufträge die KI zwischen den beiden Gruppen erfindet.

10. Der Nachtrags-Modus (Neuzugänge & Erinnerungen)

Kommt eine Gruppierung dazu (wird aktiv gesetzt), steigt das **Soll** pro Gruppe um 1. Alle, die vorher „vollständig“ (grün) waren, stehen dann bei „12/13“ — die fehlende Bewertung ist genau die über den Neuzugang. Das ist **kein Fehler**, sondern korrekt: Ihnen fehlt tatsächlich eine Bewertung.

Dafür gibt es das Häkchen „**Nur fehlende Bewertungen**“ beim Versand: Es postet je Gruppierung nur die Menüs, zu denen noch **keine** Antwort vorliegt.

- **Neuzugang:** bekommt alle Menüs; alle anderen genau eines (für den Neuzugang).
- **Erinnerung:** Wer 5 von 12 abgegeben hat, bekommt gezielt die fehlenden 7.
- Wer vollständig ist, wird mit Grund „bereits vollständig“ übersprungen.

Niemand klickt dadurch etwas doppelt, und die vorhandenen Antworten bleiben.

11. Aufräumen & Löschen

Erhebungs-Antworten (**relation_proposals**)

- **X neben einer Bewertung** — eine Richtung löschen, Gegenrichtung bleibt.
- **„Löschen“ am Zeilenende** — beide Richtungen eines Paares.
- **„Antworten dieser Gruppierung löschen“** (Rücklauf-Karte) — alles, was eine Gruppierung abgegeben hat; sie kann neu bewerten, ohne dass andere von vorn anfangen.
- **„Alle Einschätzungen löschen“** — der gesamte Rücklauf.

All das lässt **crew_relations** **unangetastet** — nur die Erhebung wird geleert. Der Hinweis steht auch im Bestätigungsdialog, weil das sonst die naheliegende Sorge wäre.

Discord-Nachrichten (**survey_messages**)

Die Menüs hängen an der Discord-Nachricht, nicht an der Erhebung — eine alte Umfrage bleibt bedienbar, solange ihre Nachricht im Channel steht. Damit man nicht versehentlich über eine alte Fassung abstimmt:

- „Umfrage im Channel entfernen (n)“ (pro Gruppierung) und
- „Alle Umfragen aus Discord entfernen (n)“ (alle Channels).

Der Versand merkt sich dafür Channel + Message-ID jeder geposteten Nachricht. Die **abgegebenen Antworten bleiben** — die liegen in der Datenbank, nicht in der Nachricht.

12. API-Referenz

Alle Endpunkte unter `/api/relations/survey`, Admin-Auth (Basic).

Methode	Pfad	Zweck
POST	<code>/send</code>	Umfrage versenden (mit <code>only_missing</code> , Frist)
GET	<code>/status</code>	Rücklauf je Gruppierung + <code>soll_pro_gruppe</code>
GET	<code>/matrix</code>	Gegenüberstellung: beide Richtungen, Befund, <code>current</code> , <code>current_notes</code> , <code>vorschlag</code>
PUT	<code>/proposal</code>	eine Meldung setzen/nachtragen (Upsert, gerichtet)
DELETE	<code>/proposal?from_crew_id=&to_crew_id=</code>	eine Richtung löschen
DELETE	<code>/pair?a_id=&b_id=</code>	beide Richtungen eines Paares
DELETE	<code>/crew/{crew_id}</code>	alle Antworten einer Gruppierung
DELETE	<code>/reset</code>	alle Erhebungs-Antworten
POST	<code>/ai-suggest</code>	KI-Vorschlag für ein Paar (schreibt nichts)
POST	<code>/finalize</code>	geltenden Stand + Notiz nach <code>crew_relations</code> (der einzige Schreibzugriff auf die geltende Matrix)
DELETE	<code>/messages/{crew_id} · /messages</code>	Discord-Umfrage-Nachrichten entfernen

Bot-Endpunkt (intern, HTTP-API auf `:8001`):

Methode	Pfad	Zweck
POST	<code>/post_relation_survey</code>	Nachricht + Auswahlmenüs in einen Channel posten